

Third-Party Services

What Velora needs you to sign up for, why the backend needs it, and how to set each one up.

Velora backend · MVP 1 · Prepared 1 August 2026 · Reflects the codebase at commit `8b8d3b6`

oo Summary

Service	Purpose	Recommended	Cost to start	When
Database	Postgres	Neon	Free	DONE
Image hosting	Portfolio photos	Cloudinary	Free	NOW
Email	Password reset	Resend	Free	NOW
Push notifications	Booking & quote alerts	Expo Push	Free	BEFORE USERS
API hosting	Run the backend	Railway	~\$5/mo	BEFORE USERS
Domain	Email sender + API URL	Namecheap / Cloudflare	~₹900/yr	BEFORE USERS
Apple Developer	Ship to iOS + APNs	—	\$99/yr	BEFORE USERS
Google Play	Ship to Android	—	\$25 once	BEFORE USERS
Error monitoring	Catch crashes	Sentry	Free	OPTIONAL
Payments	Collect money	Razorpay	Free + 2%/txn	LATER PHASE
SMS / OTP	Phone verification	MSG91	Pay-as-you-go	OPTIONAL

Total to get started properly: ₹0. Cloudinary, Resend, Expo and Sentry all have free tiers that comfortably cover development and early users. Real money only starts at hosting (~\$5/mo) and the store accounts (\$99/yr + \$25 once), and only when you ship.

Do these two first. Cloudinary and Resend are the only ones blocking backend work right now — everything else can wait until the app exists. Both take about ten minutes to sign up for.

o1 Cloudinary — image hosting NOW

WHY THE BACKEND NEEDS IT

A portfolio item stores `coverImageUr1` as a plain string. There is no upload endpoint, so today a professional can only reference an image that already exists somewhere on the internet. The upload screen in the app has nowhere to put the file until this exists.

WHY CLOUDINARY OVER THE ALTERNATIVES

It resizes and crops on the fly from the URL. A portfolio grid needs thumbnails, a detail screen needs a large version, and phone cameras produce 4 MB photos — with Cloudinary that is a URL parameter, with plain object storage it is a job you have to build and run.

Alternative	Free tier	Trade-off
Cloudinary (recommended)	25 credits/mo (~25k transforms)	Transforms included; generous free tier
Supabase Storage	1 GB	Simpler, but no image transforms
AWS S3 + CloudFront	12 months free	Cheapest at scale, most setup, no transforms
UploadThing	2 GB	Very easy, but a thin wrapper you'd outgrow

SETUP

1. Sign up at `cloudinary.com` — the free "Programmable Media" plan, no card required.
2. The dashboard shows **Cloud name**, **API key** and **API secret**. Copy all three.
3. Settings → Upload → Upload presets → add a preset named `velora_portfolio`. Set **Signing mode: Signed** and a folder such as `velora/portfolio`.
4. Optionally add an incoming transformation on the preset to cap uploads at ~2000px wide — this stops a 12 MP phone photo from eating your quota.
5. Put the credentials in `.env` (never in git):

```

CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=123456789012345
CLOUDINARY_API_SECRET=your-secret
CLOUDINARY_UPLOAD_PRESET=velora_portfolio

```

Signed, not unsigned. Cloudinary also offers unsigned presets, where the app uploads with no server involvement. That means anyone who reads your app bundle can upload anything to your account, at your cost. The backend signs each upload instead — the setup below is built for that.

What I build once you have this: `GET /api/uploads/signature` — a `PROFESSIONAL` -only endpoint returning a short-lived signature. The app uploads the file straight to Cloudinary and sends the resulting URL to `POST /api/portfolio`. The image never passes through your server, so uploads cost you no bandwidth and no memory. *Roughly 2 hours.*

02 Resend — email delivery NOW

WHY THE BACKEND NEEDS IT

Password reset is fully built *except* delivery: token generation, hashing, 30-minute expiry, single use, and revoking every session on reset all work and are tested. The only implementation of `PasswordResetNotifier` writes the token to the application log, which is fine in development and unacceptable in production — a reset token in a log file is a working reset token for anyone who can read logs.

Alternative	Free tier	Trade-off
Resend (recommended)	3,000/mo, 100/day	Cleanest API, plain HTTPS from Java, no SMTP setup
Brevo	300/day	Strong India deliverability; clunkier dashboard
AWS SES	~\$0.10 per 1,000	Cheapest at scale; sandbox removal needs a support ticket
SendGrid	100/day	Fine, but heavier and stricter onboarding

SETUP

1. Sign up at `resend.com`.

2. **To start:** use their shared test domain — you can send to your own address immediately with no DNS work. Enough to build and test the whole flow.
3. **Before real users:** add your domain, then add the DKIM/SPF records Resend gives you at your registrar. Without this, reset emails land in spam.
4. API keys → create one with **Sending access** only.
5. Add to `.env` :

```
RESEND_API_KEY=re_XXXXXXXXXXXXX
MAIL_FROM_ADDRESS=noreply@yourdomain.com
MAIL_FROM_NAME=Velora
# Deep link the reset email points at; the app opens this with the token
APP_PASSWORD_RESET_URL=velora://reset-password
```

What I build once you have this: a `ResendPasswordResetNotifier` bean plus the email template. It replaces the logging one automatically — no other code changes, because delivery was deliberately kept behind an interface. *Roughly 1 hour.*

03 Push notifications BEFORE USERS

WHY IT MATTERS

The whole product is asynchronous: a customer books and waits for an admin to assign someone; a professional waits for bookings; a customer waits for a quotation. With no push, every one of those handoffs depends on the user happening to reopen the app. Nothing in the backend touches notifications yet.

If you build with Expo, use Expo Push. It is free, handles both iOS and Android through one API, and spares you certificate handling. Only drop to raw FCM + APNs if you leave Expo.

SETUP (EXPO ROUTE)

1. Create an Expo account, then an EAS project for the app.
2. The app requests notification permission and gets an Expo push token.
3. The app sends that token to the backend, which stores it per user and device.
4. Backend sends to Expo's HTTP API when something happens. No Apple/Google credentials needed for Android; iOS still needs the Apple Developer account for a real device build.

```
EXPO_ACCESS_TOKEN=your-expo-token # only needed for higher rate limits
```

What I build: a `device_tokens` table + register/unregister endpoints, and a notification service hooked into the events that matter — booking assigned, booking confirmed, quotation sent, quotation accepted/rejected, booking completed. *Roughly 1 day.*

04 Hosting the API BEFORE USERS

The backend currently runs on your laptop. A phone on the same Wi-Fi can reach it by LAN IP for early testing, but anything beyond that — a tester elsewhere, a TestFlight build, the store — needs it hosted. Neon already hosts the database, so this is only the Spring Boot app.

Option	Cost	Trade-off
Railway (recommended)	~\$5/mo	Deploys from GitHub, reads your env vars, minimal config
Render	Free tier / \$7/mo	Free tier sleeps after inactivity — first request takes ~30s
Fly.io	~\$5/mo	Good India-region latency; more Docker knowledge needed
AWS / GCP	Varies	Only worth it once you have a reason

SETUP

1. Connect the GitHub repo. Railway detects Maven and Java 21 on its own.
2. Copy every variable from `.env` into the platform's environment settings — **generate a fresh** `JWT_SECRET` **for production**, do not reuse the dev one.
3. Set `CORS_ALLOWED_ORIGINS` to your app's origins.
4. Flyway applies all 10 migrations on first boot automatically.
5. Point a subdomain such as `api.yourdomain.com` at it.

05 Domain & store accounts BEFORE USERS

Domain — ~₹900/year

Needed for two things: a credible email sender address (`noreply@yourdomain.com` , which also fixes deliverability) and a stable API URL. Namecheap or Cloudflare Registrar; Cloudflare sells at cost and includes DNS.

Apple Developer Program — \$99/year

Required to put an iOS build on any real device, including your own for testing, and to use APNs for push. **Start this early** — enrolment verification can take several days, and for a company account Apple asks for a D-U-N-S number, which takes longer still. An individual account is faster and upgradeable later.

Google Play Console — \$25 once

One-time fee, approval usually within a day or two. Android testing works without it via a direct APK, so this is only needed to publish.

06 Optional and later

Sentry — error monitoring OPTIONAL

Free tier covers 5,000 errors/month. Worth adding before launch: the 124 automated tests cover business rules, but nothing tells you when a real user hits a 500 in production. Add the SDK to both backend and app; setup is roughly 20 minutes.

```
SENTRY_DSN=https://xxxx@sentry.io/xxxx
```

Payments LATER PHASE

Nothing about payments exists in the backend — quotations end at "accepted". When that phase starts: **Razorpay** for an Indian audience (UPI, cards, netbanking, INR settlement; ~2% per transaction), **Stripe** only if you go international. Both

need business verification — a GST number and bank account for Razorpay — so allow a week before you can accept real money.

SMS / OTP OPTIONAL

Only if you decide phone numbers must be verified. **MSG91** is the cheaper India option; Twilio is the global default and costs noticeably more per SMS. Not required by anything currently built — accounts are email-based.

07 Handling credentials

Never commit a real key. `.env` is already git-ignored, and `.env.example` holds only placeholder names. Every service above issues keys you can revoke and reissue — if one is ever pasted into a chat, a screenshot, a commit or a support ticket, revoke it rather than hoping.

Use separate keys per environment where the provider allows it (Cloudinary and Resend both do), so a leaked development key can be killed without taking production down. Everything the backend reads is listed in `.env.example`; add these as you sign up:

```
# Cloudinary
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
CLOUDINARY_UPLOAD_PRESET=velora_portfolio

# Email (Resend)
RESEND_API_KEY=
MAIL_FROM_ADDRESS=noreply@yourdomain.com
MAIL_FROM_NAME=Velora
APP_PASSWORD_RESET_URL=velora://reset-password

# Push (Expo) - later
EXPO_ACCESS_TOKEN=

# Monitoring - optional
SENTRY_DSN=
```

08 Suggested order

#	Do this	Time	Unblocks
1	Cloudinary signup + signed preset	10 min	Portfolio upload, end to end
2	Resend signup (test domain is fine)	10 min	Password reset actually works
3	Start Apple Developer enrolment	15 min + wait	iOS device builds and push; the wait is why it's early
4	Buy the domain	15 min	Email deliverability, API URL
5	Expo account	5 min	Push, once the app shell exists
6	Railway + Google Play	1 hour	Testers on real devices

Steps 1 and 2 are the only ones with backend work waiting behind them. Give me the credentials in `.env` — not pasted anywhere else — and both integrations get written the same day.

Free tiers, prices and dashboard wording were accurate when this was written and do change; treat the figures as indicative and confirm on each provider's pricing page before committing.